

A PERSONAL DEEP-DIVE INTO AGENTIC ENGINEERING

Code to Cloud, Autonomously.

How AI Agents Transform the Entire SDLC —
From Epic to Deployment, Manually and at Scale.

Phase 1: Manual SDLC

Phase 2: Agentic Orchestra

Phase 3: Legacy Intelligence

✦ *GitHub Copilot Compatible* ✦ *Claude Code Compatible* ✦

Designed by Curiosity. Built by Discipline. Delivered by Nirav Mehta.

Why Our SDLC Needs to Evolve

The same manual process that took 3 weeks in 2018 still takes 3 weeks today. These six pain points block every team.



Spike Before Every Story

Devs spend 1–3 days reading legacy code before writing a story. Multiply by 15+ services and 4 sprints a year.



Zero Documentation

5–8 year-old microservices with no docs, no architecture records. Every new developer starts from zero.



Manual Every Step

Epic → Stories → Design → Code → Tests → Deploy. Each handoff is a context switch, a meeting, a delay.



No Contract Safety

Customer-facing APIs break silently. No automated contract tests. A routine refactor becomes a production incident.



Questions Slow Work

'Who owns this table?' 'Is it safe to change this method?' — each question costs hours before work can begin.



Deployment Anxiety

No defined deployment plan, no rollback playbook, no smoke test automation. Every GKE deploy is a manual risk.

Three Phases — One Journey to Autonomous SDLC

Start with full control. Grow into full autonomy. Each phase is independently valuable — no big bang adoption.

PHASE ONE

Manual SDLC with AI Agents

1

Every agent runs when you prompt it. Perfect for learning, regulated environments, or sensitive features.

- ▶ Jira Epic → Story Agent
- ▶ Architect → Planner → Dev Agent
- ▶ Test Agent → DevOps Agent → GKE
- ▶ *Works in Copilot & Claude*

Low Risk • Full Control

PHASE TWO

Queen Agent Orchestration

2

The Queen Agent auto-orchestrates all SDLC phases. You approve at 3 gates. It handles everything else.

- ▶ Epic → Stories → Code → Tests → GKE
- ▶ Auto token optimisation
- ▶ Full audit trail
- ▶ *Copilot + Claude compatible*

High Speed • Human Gates

PHASE THREE

Legacy Code Intelligence

3

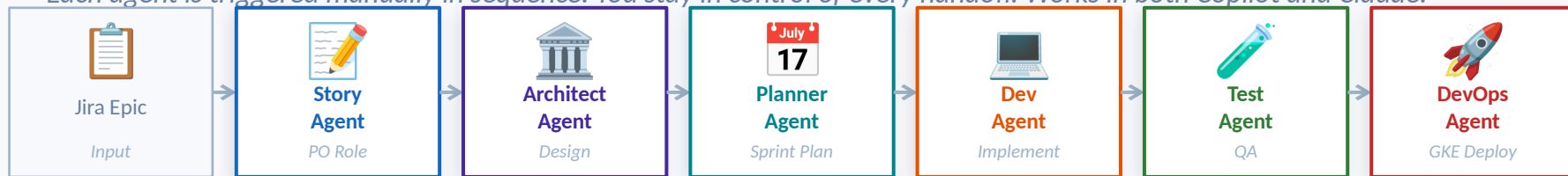
For 5–8 year-old services. Scan, understand, document, generate stories, find bugs — before touching a line.

- ▶ Code Archaeology skill
- ▶ Story Generator from code
- ▶ Repo QA Agent (ask your code)
- ▶ *CodeFixes per finding*

Legacy-Safe • Zero Guessing

Classic SDLC — Epic to GKE, One Agent at a Time

Each agent is triggered manually in sequence. You stay in control of every handoff. Works in both Copilot and Claude.



1	Story Agent	User stories with Fibonacci estimates, acceptance criteria, risk levels, auth labels (JWT / Basic Auth via Apigee / Public)	<i>Copilot: /create-story Claude: story-helper agent</i>
2	Architect Agent	Service boundaries, API contracts (OpenAPI), data models, Apigee impact, GKE resource changes needed	<i>Both Copilot and Claude read .claude/context/apis.md + deps.md</i>
3	Planner Agent	Sprint breakdown with dependency ordering, parallel tracks (backend vs frontend), definition of done	<i>Copilot: /create-story Claude: .claude/agents/story-helper.md</i>
4	Dev Agent	Code + PRs. Reads CodeFixes/ for known issues. Runs mvn test after each story. Retries 3x on failure.	<i>Claude Code (primary) Copilot inline suggestions follow copilot-instructions.md</i>
5	Test Agent	JUnit tests (matching your JUnit version), @WebMvcTest with Apigee headers, Playwright E2E (UI + API + Customer contracts)	<i>Both: .claude/context/tests.md defines JUnit version, base class, H2 vs TestContainers</i>
6	DevOps Agent → GKE	Dockerfile, K8s manifests, GitHub Actions CI/CD, GKE_READINESS checklist, blue-green deploy + smoke tests	<i>Both: .claude/context/gke.md Copilot: /review-change on manifests</i>

Manual AI-Assisted SDLC — Benefits and Constraints

The right choice for teams starting their AI journey or working in highly regulated environments

✓ Benefits

- ✓ Full human control at every step
- ✓ Easy to correct if agent misunderstands
- ✓ No learning curve — use Copilot or Claude, your choice
- ✓ Works with existing Jira and GitHub processes
- ✓ Great for onboarding — reasoning is fully visible
- ✓ Ideal for regulated or highly sensitive work

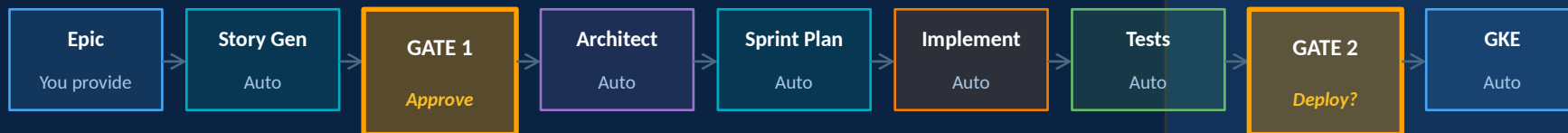
⚠ Constraints

- Requires human to trigger each step manually
- Context can be lost between agent sessions
- Missed steps mean incomplete or wrong output
- No token optimisation across phases
- Cannot run overnight or unattended
- Phase 2 solves all of these — when you're ready

The Queen Agent Orchestrates Everything

You provide the Epic. Queen handles every phase — autonomously, with token optimisation, pausing only at human gates.

Works with any Claude-compatible or Copilot Agent runner.



Reasons Between Phases



Reads previous agent output before starting next. Consistent context, no dropped info across the entire pipeline.



Token Optimisation



Loads only the context files each phase needs. 10x fewer tokens than re-reading source code every question.



3 Human Gates Only



Story approval, architecture approval, deploy approval. Everything else runs unattended — including overnight.

Result: Epic → Deployed app in ~3-4 hours. Your involvement: 3 approvals, ~30 minutes. Works in Copilot Agent + Claude Code.

What Each Agent Does in the Orchestra

Six specialised agents + one Queen. Each has one job. All run in GitHub Copilot Agent mode or Claude Code.



Queen Agent

Orchestrator

Reads pipeline state. Decides next step. Passes output between agents. Invokes human gates. Never guesses — always reasons from context.

→ Complete SDLC delivery



Story Agent

Product Owner

Reads Epic + SERVICE_PROFILE + auth.md.
Generates user stories with Fibonacci estimates, acceptance criteria, auth labels (Apigee/JWT), risk levels.

→ Type A + B Jira stories



Architect Agent

Solution Architect

Reads stories + deps.md. Designs service boundaries, API contracts, data models. Checks Apigee and GKE impact against existing patterns.

→ ARCHITECTURE.md



Dev Agent

Senior Dev

Implements story by story. Reads CLAUDE.md + CodeFixes. Follows exact patterns. Runs mvn test after each story. Retries on failure (3x).

→ Code + PRs



Test Agent

QA Engineer

Writes JUnit tests (matching your JUnit version), controller tests with Apigee headers, Playwright E2E (UI + API + customer contracts). 80% coverage target.

→ Full test suite



DevOps Agent

Platform Engineer

Generates K8s manifests, GitHub Actions pipeline, GKE_READINESS checks, smoke tests. Rollback on failure. Blue-green deploy.

→ Live on GKE

Worked Example: Add Order Status Notifications

Epic to GKE — showing every agent handoff. Runs in Copilot Agent mode or Claude Code autonomously.

T+0m	You	Drop Epic into Queen Agent	"Add push notifications when mortgage order status changes. CustomerA integration required."
T+5m	Story Agent	Reads docs, generates stories	8 stories generated. US-E001 [Existing] Order Status API — 5pt HIGH JWT. US-G001 [Add] Notification integration — 8pt P1.
T+12m	👑 GATE 1	You approve stories	"14 stories, 52 story points. Confirm to proceed to architecture?"
T+15m	Architect	Reads deps.md + apis.md	Designs: order-service → RabbitMQ → notification-service. No schema change needed. JWT Pattern B.
T+45m	Dev Agent	Implements story by story	Creates NotificationService, OrderEventPublisher. Reads CodeFixes: FIX-003 timeout missing — applies fix first.
T+2h	Test Agent	Full test suite generated	38 JUnit tests, 82% coverage. Playwright: 4 E2E including CustomerA contract test for order webhook.
T+2.5h	👑 GATE 2	You approve deploy	"All tests pass. 82% coverage. CustomerA contract: PASS. Ready for GKE staging deploy?"
T+3h	DevOps Agent	Deploy to GKE + smoke tests	Docker build, push to Artifact Registry, kubectl apply, smoke: POST /orders webhook 

Phase 3: Four Agents for Legacy Code

Eliminates the spike story bottleneck. Scan → Document → Ask → Fix → Generate Stories. All in one session.

⚠ Reality: Your legacy services have years of tribal knowledge locked in undocumented Java. Phase 3 gives you institutional memory before any change — and eliminates spike stories entirely.



Code Archaeology

Copilot + Claude

Scans the entire repo. Writes 17+ small focused files — one concern per file, under 80 lines each. Works in both Copilot Chat (@workspace) and Claude Code.

- ▶ SERVICE_PROFILE.md — what this service is
- ▶ auth.md — Apigee + JWT model
- ▶ gke.md — K8s config snapshot
- ▶ deps.md — inter-service blast radius
- ▶ tests.md — JUnit version, coverage state



Story Generator

Eliminates Spike Stories

Reads archaeology docs and generates stories directly from CODE — no spike story needed. Type A (existing behaviour) + Type B (gaps). Run from Copilot /create-story or Claude story-helper agent.

- ▶ Type A: existing endpoints as stories
- ▶ Type B: P1 security gaps, P2 coverage
- ▶ Fibonacci estimates from code complexity
- ▶ Risk level from deps.md contracts
- ▶ Jira-ready format with auth labels



Repo QA Agent

Like NotebookLM for Your Code

Answers any developer question from the context docs — no re-scanning. Ask from Copilot Chat or Claude. Works inside your IDE, not an external tab. Your code never leaves your workspace.

- ▶ 'Is it safe to change X?' → risk level
- ▶ 'How does auth work?' → Pattern A/B
- ▶ 'How do I write a test?' → JUnit pattern
- ▶ 'Story points for X?' → estimate
- ▶ 'What's broken in GKE?' → ❌ checks



CodeFixes Folder

Exact Fixes, Exact Lines

One file per risky finding. Contains exact file path, line number, current code block, suggested replacement code, and a 'why this is safe' section. Never auto-applies — developer reviews each.

- ▶ FIX-001: Missing @Valid (P1 Security)
- ▶ FIX-002: JVM OOM risk (P1 GKE)
- ▶ FIX-003: No HTTP timeout (P2)
- ▶ FIX-NNN: ...one file per finding
- ▶ _INDEX.md: all findings by severity

From Spike Story → Code Analysis → Story Generation

The workflow your team runs today — replaced. No spike, no manual analysis, no assumption-based stories.

BEFORE (Manual Spike — 2-3 days)

- Dev assigned spike story
- 2 days reading 50k lines of code
- Ask 3 engineers for context
- Write rough notes in Confluence
- PO writes stories from the notes
- Stories miss edge cases + auth risks
- Sprint estimate is a guess

 **Total: 2-3 days, still incomplete**

AFTER (Code Archaeology + Story Generator — 30 min)

- ▶ Open service in VS Code (or terminal)
- ▶ Run Code Archaeology (15 min, read-only)
- ▶ 6 tiny context files written to repo
- ▶ Ask Repo QA: 'what does this service do?'
- ▶ Ask Repo QA: 'is it safe to change X?'
- ▶ Run Story Generator: 'create stories from docs'
- ▶ Stories include auth model, risk, estimates

 **Total: 30 min, production-accurate**

Time saved per sprint: ~2 days per service → ~40 dev-days saved per year for a 10-service team

Repo QA Agent vs Google NotebookLM

Repo QA lives in your IDE. Reads your actual code. Your data never leaves your workspace.

Feature	Repo QA Agent (Copilot + Claude)	Google NotebookLM
Where it lives	In your IDE / same workspace	External browser tab
Code repository access	Direct — reads your actual files	Cannot upload Git repos
Auth model awareness	Knows your Apigee + JWT patterns	Generic — no context
Answer source	From scanned code (accurate)	From uploaded docs only
Tool compatibility	GitHub Copilot + Claude Code	No IDE integration
CodeFixes integration	Links to exact fix files	No code fix capability
Update on code change	Re-run archaeology, docs update	Manual re-upload every time
Custom agents / prompts	3 sub-agents, 4 slash commands	No custom agents
Privacy / data residency	Never leaves your environment	Data goes to Google servers

Key advantage: Your code never leaves your workspace. Ask 'how does auth work?' — the answer comes from YOUR actual code, not a generic document.

One Scan. Both Tools Trained. 17+ Small Files.

Archaeology writes into both tools' native agent locations — one source of truth, zero duplicate maintenance.

repo-root/

CLAUDE.md

Claude auto-reads

AGENTS.md

Both + Cursor

.claude/context/

stack · patterns · auth · gke

apis · deps · tests

.claude/agents/

archaeologist · reviewer · story-helper

.github/

copilot-instructions.md

Copilot auto-reads

prompts/ → /new-endpoint /write-test

prompts/ → /review-change /create-story

CodeFixes/

_INDEX.md FIX-001.md FIX-002.md ...

How Tools Connect

GitHub Copilot

Reads copilot-instructions.md → loads context/*.md → slash commands

Claude Code

Reads CLAUDE.md → loads context/*.md on demand → sub-agents

AGENTS.md

Shared open standard — Copilot + Claude + Cursor/Windsurf all read this



One Source of Truth

Update auth.md once → both Copilot and Claude give the same answer.

Each file under 80 lines. Load only what you need.
10x fewer tokens per session vs re-scanning source code.

CodeFixes/ — Exact Findings, Exact Fixes

Not vague warnings. One file per issue: actual file path, line number, current code, suggested replacement, why it's safe

P1 Security

- Missing @Valid on customer endpoints
- Basic Auth parsed in service (not at Apigee)
- Hardcoded secret in application.yml
- PII (email/phone) in plain log statements

P1 GKE Reliability

- JVM -Xmx > 75% of container limit (OOM risk)
- Missing liveness/readiness probes
- 'latest' image tag in production
- No PodDisruptionBudget configured

P2 Code Correctness

- N+1 query in JPA relationship fetch
- @Transactional missing on multi-write service
- HTTP client call with no timeout set
- Swallowed exception (empty catch block)

BENEFITS

Concrete, Measurable Impact

Six metrics that compound over time across the team

Spike Story Time Saved

Before	After
2-3 days	30 min

Saving: ~95%

Epic to First Code

Before	After
2-3 weeks	4-8 hours

Saving: ~90%

Test Coverage Starting Point

Before	After
0% (no docs)	60%+ generated

Saving: +60pt

New Dev Onboarding

Before	After
2 weeks	1 hour

Saving: ~95%

Deploy Safety

Before	After
Manual checklist	Automated gates

Saving: Zero guesses

Customer API Breakage Risk

Before	After
Unknown	Contract tested

Saving: Prevented

Three Strategic Takeaways

Real impact. Immediate applicability. No specialised hardware required.

01 Speed Without Sacrifice

Manual AI-assisted teams deliver stories 60% faster with higher quality. Queen Agent orchestration compresses a 3-week sprint to 4–8 hours for new features — while maintaining all quality gates and human approval checkpoints.

02 Legacy Risk Eliminated

Code Archaeology gives institutional memory for services built 5–8 years ago. Before any change: we know which customers are impacted, what the blast radius is, what will break. No more production incidents from 'harmless refactors'.

03 Team Upskilling Built In

New team member gets up to speed on any service in 1 hour instead of 2 weeks. Repo QA answers questions from your own code, in your IDE, using both Copilot and Claude. Institutional knowledge that compounds over time.

Start This Week — 3 Actions

No special hardware. No new licences beyond what you already have. Just Claude Code + VS Code + your repos.

1

This Week Code Archaeology on One Service

1

Install Claude Code: `npm install -g @anthropic-ai/claude-code`

2

Open your most critical service in VS Code

3

Say: 'Scan this repo — Spring Boot on GKE via Apigee'

4

17+ context files written for both Copilot and Claude

5

Copilot and Claude Code both trained — same session

✓ Both tools trained in 30 min

2

This Sprint Story Generation from Code

1

After archaeology, paste your next Epic

2

Run story-generator skill

3

Get Type A + B stories with estimates

4

Compare with manual stories — check accuracy

5

Create Jira tickets from output

✓ Stories in under 1 hour of human time

3

This Quarter Phase 2 Queen Orchestration

1

Apply archaeology to all 8–10 services

2

Test Queen Agent on a non-critical new feature

3

Run full pipeline: Epic → GKE in 3–4 hours

4

Establish the 3-gate approval process

5

Scale to the full team

✓ Full agentic SDLC running on your team

"Stop Reading Code. Start Asking It Questions."

The agentic SDLC doesn't replace engineers — it removes the hours of archaeology, documentation, and context-switching
so engineers can focus on what only humans can do: judgment, creativity, and care.



Code Archaeology:
understand before you change



Story Generator:
stories from real code



Queen Agent:
let the orchestra play

Questions & Discussion